

Task 3

Write a stored procedure `sp_enroll_student` (MySQL) or function `fn_enroll_student` (PostgreSQL) that accepts `student_id`, `course_id`, and `enrollment_date`, checks for duplicate enrollment, and inserts the record.

```
328
329 DELIMITER $$
330 • CREATE PROCEDURE sp_enroll_student(IN p_student_id INT,IN p_course_id INT,IN p_enrollment_date DATE)
331 BEGIN
332     DECLARE v_count INT;
333     SELECT COUNT(*)
334     INTO v_count
335     FROM enrollments
336     WHERE student_id = p_student_id AND course_id = p_course_id;
337     IF v_count > 0 THEN
338         SELECT 'Student is already enrolled in this course.' AS Message;
339     ELSE
340         INSERT INTO enrollments(student_id, course_id, enrollment_date)
341         VALUES (p_student_id, p_course_id, p_enrollment_date);
342         SELECT 'Enrollment successful.' AS Message;
343     END IF;
344 END $$
345
346 DELIMITER ;
```

Output			
Action Output			
#	Time	Action	Message
✓ 14	16:50:46	SELECT * FROM vw_student_enrollment_summary LIMIT 0, 1000	4 row(s) returned
✓ 15	16:50:58	UPDATE vw_student_enrollment_summary SET enrollment_year = 2024 WHERE student_id = 4	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 16	16:51:02	SELECT * FROM vw_student_enrollment_summary LIMIT 0, 1000	4 row(s) returned
✓ 17	17:10:19	CREATE PROCEDURE sp_enroll_student(IN p_student_id INT, IN p_course_id INT, IN p_enrollment_date...	0 row(s) affected

```
347 • call sp_enroll_student(1,1,'2026-07-02');
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Message			
▶ Student is already enrolled in this course.			

45. Write a procedure `sp_transfer_student` that moves a student from one department to another. Wrap the UPDATE and a log-insert into a new table

department_transfer_log inside a single transaction.
ROLLBACK if either statement fails.

```
DELIMITER $$

CREATE PROCEDURE sp_transfer_student(
    IN p_student_id INT,
    IN p_new_department INT
)
BEGIN
    DECLARE v_old_department INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;

    START TRANSACTION;

    SELECT department_id
    INTO v_old_department
    FROM students
    WHERE student_id = p_student_id;

    UPDATE students
    SET department_id = p_new_department
    WHERE student_id = p_student_id;

    INSERT INTO department_transfer_log(
        student_id,
        old_department_id,
        new_department_id,
        transfer_date
    )
```

```
VALUES (
    p_student_id,
    v_old_department,
    p_new_department,
    NOW()
);
```

```
COMMIT;
END $$
```

```
DELIMITER ;
```

```
6  \ ENDU $$
7
8  DELIMITER ;
9  • CALL sp_transfer_student(1,2);
0  • SHOW CREATE PROCEDURE sp_transfer_student;
```

Procedure	sql_mode	Create Procedure	character_set_client	collation_connection	Database Collation
sp_transfer_student	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER='root'@'localhost' PROC...	utf8mb4	utf8mb4_0900_ai_ci	utf8mb4_0900_ai_ci

Test the transaction by manually introducing an error (e.g., invalid foreign key) and verify that the first UPDATE is also rolled back.

```
401
402 • CALL sp_transfer_student(1,999);
403 • SELECT student_id, department_id
404 FROM students
405 WHERE student_id = 1;
```

student_id	department_id
1	2
NULL	NULL

Use SAVEPOINT to create a mid-transaction checkpoint: insert two enrollment records; set a SAVEPOINT after the first; deliberately fail the second; ROLLBACK TO SAVEPOINT and verify only the first record was saved.

```
SELECT * FROM enrollments;

START TRANSACTION;
INSERT INTO enrollments
(student_id, course_id, enrollment_date, grade)
VALUES
(3,2,'2026-07-02','A');

SAVEPOINT first_insert;

INSERT INTO enrollments
(student_id, course_id, enrollment_date, grade)
VALUES
(3,2,'2026-07-02','A');
```